Table 21: The estimated energy consumption of the operations and computing elements applied in the proposed unsigned and signed multipliers [4, 5, 22, 28].

Operation/ Component	Energy Consumption (nJ)	Operation/ Component	Energy Consumption (nJ)	Operation/ Component	Energy Consumption (nJ)
FALSE	0.05	NOT/Proposed Signed PPU_3	0.13	AND	0.33
NAND	0.24	OR	0.244	XOR	0.374
XNOR	0.9	First MUX 2:1	0.6	Second MUX 2:1	0.9
Half Adder [28]	1.02	Full Adder [23]	1.85	4:2 Compressor [28]	3.76
Classic Unsigned PPU_1	1.68	Classic Unsigned PPU_2	2.18	Classic Unsigned PPU_3	2.51
Classic Signed PPU_1	1.68	Classic Signed PPU_2	1.59	Classic Signed PPU_3	1.02
Classic Signed PPU_4	2.18	Classic Signed PPU_5	2.33	Classic Signed PPU_6	2.09
Classic Signed PPU_7	2.42	Classic Signed PPU_8	1.85	Proposed Unsigned PPU_1	1.602
Proposed Unsigned PPU_2	2.156	Proposed Unsigned PPU_3	2.5	Proposed Signed PPU_1	1.602
Proposed Signed PPU_2	1.62	Proposed Signed PPU_4	2.156	Proposed Signed PPU_5	2.5
Proposed Signed PPU_6	2.15	Proposed Signed PPU_7	2.475	Proposed Signed PPU_8	0.74

$$E_{UAmultiplier} = \#PPU_1 \times E_{PPU1} + \#PPU_2 \times E_{PPU2} + \#PPU_3 \times E_{PPU3} + \#HA \times E_{HA} + \#FA \times E_{FA} + \#AND \times E_{AND} \quad (26)$$

$$E_{SAmultiplier} = \#PPU_1 \times E_{PPU1} + \#PPU_2 \times E_{PPU2} + \#PPU_3 \times E_{PPU3} + \#PPU_4 \times E_{PPU4} + \#PPU_5 \times E_{PPU5} + \#PPU_6 \times E_{PPU6} + \#PPU_7 \times E_{PPU7} + \#PPU_8 \times E_{PPU8} + \#FA \times E_{FA} + \#AND \times E_{AND} \quad (27)$$

Table 22 compares the proposed serial unsigned array multiplier with other classic serial unsigned multipliers regarding the number of memristors, computational steps, and energy consumption. Furthermore, the proposed serial signed array multiplier is evaluated and compared with the classic signed multipliers, and the results are reported in Table 23. The same method (see (12) and (25)–(27)) is applied to compute the number of computational steps and energy consumption of classic multipliers. The authors tried to reduce the number of applied work memristors to its minimum in the serial unsigned and signed array multipliers to calculate the number of required memristors.

Table 22: The comparison of IMPLY-based serial unsigned multipliers.

Multiplier	No. of Steps			No. of Memristors			Energy Consumption (nJ)		
	Total	n=8	%Imp.	Total	n=8	%Imp.	Total	n=8	%Imp.
Dadda	$27n^2 - 32n$	1472	26.25	$n^2 + 2$	66	-	$2.18n^2 - 2.68n$	118.08	29.7
[28]	$27n^2 - 32n$	1472	26.25	$n^2 + 2$	66	-	$2.21n^2 - 2.8n - 0.05$	119	29.15
Add & Shift	$31n^2 + n + 4$	1996	-	$3n + 5$	29	56.07	$2.623n^2 - 0.023n + 0.26$	167.95	-
Array Multiplier (Classic)	$27n^2 - 32n$	1472	26.25	$5n - 4$	36	45.46	$2.18n^2 - 2.68n$	118.08	29.7
Array Multiplier (Proposed)	$25n^2 - 32n + 2$	1346	32.57	$5n - 4$	36	45.46	$2.156n^2 - 2.672n - 0.022$	116.59	30.58

According to the results of Table 22, the proposed unsigned and signed array multipliers compute the final result in fewer cycles than SOA. The number of computational steps of the proposed 4-bit unsigned array multiplier is reduced by 46% compared to the classic 4-bit add & shift multiplier and by 10% compared to

the other three classic serial multipliers. In the 8-bit architecture, the proposed unsigned array multiplier reduces the number of computational steps by at least 9% (compared to classic unsigned array and Dadda multipliers and the proposed multiplier in [28]) and a maximum of 36% (compared to the unsigned add & shift multiplier). The most energy-efficient architecture among the unsigned serial multipliers investigated in this article is the proposed array multiplier. Due to the reduction of computational steps and memristors involved in implementing the multiplier's structure, an improvement of 1.5%–44% in energy consumption has been achieved compared to classic designs (4-bit and 8-bit architectures). The proposed 4-bit serial unsigned multiplier requires the least number of memristors to compute the final result as it is improved by 6%–11% compared to classic unsigned multipliers. The classic 8-bit unsigned add & shift multiplier requires only 29 memristors for its implementation, which is the least number of memristors compared to other classic unsigned multipliers and the proposed one. The proposed multiplier has improved the number of memristors required for implementing in the crossbar array structure by 47% compared to the classic unsigned Dadda multiplier and the proposed multiplier in [28].

Table 23 compares the proposed signed array multiplier with signed add & shift, radix-2 Booth, Dadda (based on the Baugh-Wooley method), and classic signed array multipliers. The proposed signed array multiplier takes the least computational steps to calculate the output results compared to other memristive serial signed multipliers. Accordingly, compared to the classic signed architectures, the proposed 4-bit and 8-bit multipliers have improved the computational steps by 6%–65% and 7%–59%, respectively. Energy consumption is another essential circuit evaluation criterion that gets the attention of designers and researchers. The proposed signed array multiplier consumes the lowest energy compared to other classic memristive signed multipliers, and the radix-2 Booth multiplier consumes the highest energy. By reducing the number of computational steps and the involved work memristors, dynamic energy consumption is improved. Thus, the proposed 8-bit multiplier reduces the energy consumption by a maximum of 57% (compared to the radix-2 Booth multiplier) and a minimum of 2% (compared to the classic signed array multiplier). The proposed architecture requires only 16 memristors to implement the 4-bit signed array multiplier, improved by 6%–33% compared to other serial multipliers in this structure. The number of required memristors of the proposed 8-bit array multiplier is 19% more than the signed add & shift multiplier. However, it is 10% and 45% less than the radix-2 Booth multiplier and the classic Dadda multiplier. The add & shift multiplier has a simple architecture, so the number of memristors in this multiplier is minimal. However, the number of computational steps and energy consumption in this multiplier is high.

Table 23: The comparison of IMPLY-based serial signed multipliers.

Multiplier	No. of Steps			No. of Memristors			Energy Consumption (nJ)		
	Total	n=8	%Imp.	Total	n=8	%Imp.	Total	n=8	%Imp.
Add & Shift	$31n^2 + 6n + 9$	2041	26.25	$3n + 5$	29	56.06	$2.623n^2 + 0.287n + 0.57$	170.74	37.47
Radix-2 Booth	$49n^2 + 15n - 4$	3252	26.25	$4n + 8$	40	39.4	$4.169n^2 + 0.804n - 0.2$	273.05	-
Baugh-Wooley (Classic)	$27n^2 - 24n + 24$	1560	-	$n^2 + 2$	66	-	$2.18n^2 + 1.84n + 1.63$	126.43	53.7
Array Multiplier (Classic)	$27n^2 - 36n + 3$	1443	26.25	$5n - 4$	36	45.46	$2.18n^2 - 2.86n + 2.03$	118.67	56.54
Array Multiplier (Proposed)	$25n^2 - 32n + 1$	1345	32.57	$5n - 4$	36	45.46	$2.156n^2 - 2.073n - 0.067$	116.29	57.41

4.2 Application-level simulation and its results

Image processing is one of the data-intensive applications that has received much attention these days and plays a significant role in daily human lives. Slipping special kernels (filters) on reference images and performing convolution operations between the pixels of the image and the kernel are applied in various applications [33]. Applying the Gaussian blur kernel is one of the methods that can be applied to reduce the noise of the reference image (see (28)).

$$\omega = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \tag{28}$$

Table 24: The number of applied operations to the 3*3 Gaussian blur kernel on an n*n pixels grayscale image.

Arithmetic Operation	No. of Operations	n=256
8-bit unsigned multiplier	$9n^2 - 36n + 36$	580644
8-bit unsigned adder	$2n^2 - 8n + 8$	129032
9-bit unsigned adder	$3n^2 - 12n + 12$	193548
10-bit unsigned adder	$2n^2 - 8n + 8$	129032
11-bit unsigned adder	$n^2 - 4n + 4$	64516
Divider	1	1

Table 25: The number of applied operations to the 3*3 edge detection kernel on an n*n pixels grayscale image.

Arithmetic Operation	No. of Operations	n=256
8-bit signed multiplier	$5n^2 - 20n + 20$	322580
9-bit signed adder	$2n^2 - 8n + 8$	129032
10-bit signed adder	$n^2 - 4n + 4$	64516
11-bit unsigned adder	$n^2 - 4n + 4$	64516

In a 3*3 Gaussian blur filter, nine unsigned multiplication operations along with several unsigned addition operations (using 8-, 9-, 10-, and 11-bit adders) should be done to apply this kernel to the image. Also, a division (shift) operation is required for this application. Table 24 reports the number of arithmetic operations required to apply the 3*3 Gaussian blur kernel on an n*n pixels grayscale image (8-bit).

Another proper kernel in image processing is written in (29). This kernel is applied to detect the edges of an image. Edge detection is one of the essential operations in the application of pattern recognition [32]. Signed multipliers are needed to detect edges based on the kernel in (29).

$$\omega = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix} \tag{29}$$

The edge detection kernel slides over the reference image, and the convolution result is stored in the corresponding pixel. This convolution operation needs five signed multipliers and some 9-, 10- and 11-bit adders. Table 25 specifies the number of multiplication and addition operations required to detect the edges of an n*n grayscale image.

The behavioral simulation results of Gaussian blur and edge detection applications are depicted in Figure 9. The number of computational steps, memristors, and energy consumption of the unsigned and signed multipliers applied for Gaussian blur and edge detection kernels are presented in Table 26.

(a) **(b)** **(c)**

Figure 9: Simulation result of: b) Gaussian blur, and c) edge detection.

The authors' main contribution is to propose memristive unsigned and signed array multipliers. Hence, only the results of the multiplication operations in Gaussian blur and edge detection applications on the reference image (see Figure 9. a) have been reported and compared. It should be noted that the number of computational steps, memristors, and energy consumption of other basic arithmetic operations (e.g., addition) are considered the same for these two image-processing applications in all architectures. According to the results of Table